

| Relazione progetto JBlackJack

Nome e cognome: Chiara Petrucci

Matricola: 2110979

Corso: Canale M-Z (presenza)

| Utilizzo dell'MVC

Il progetto segue il modello MVC (model, view e controller), per ottenere un'organizzazione più chiara e intuitiva della logica di gioco, il controllo e la gestione degli input/output a schermo e la gestione degli stati del gioco.

- Il **model** viene utilizzato per la logica del gioco (regole del blackjack e assegnazione dei punti), per la gestione dei profili, delle regole del giocatore, del mazziere e dei bot. Inoltre, il model controlla i vari game states in base allo stato attuale del gioco. Inoltre, è indipendente dalla view e dal controller.
- La **view** si occupa di tutta la componente grafica e, pertanto, permette di coordinare tutte le animazioni, i suoni e la musica.
- Il **controller** regola l'interazione tra model e view e gestisce tutti gli input da tastiera o da mouse nei vari stati del gioco.

| Utilizzo degli stream

Nel classe `ScoreBoard` è presente il metodo `updateScores()` realizzato mediante l'uso degli stream.

Il metodo aggiorna la score board alla fine di ogni partita. Quindi, ordina una lista di profili (`scores`) selezionando i primi 5 giocatori con il maggior numero di vittorie.

Nel dettaglio

Il metodo recupera tutti i profili disponibili tramite l'invocazione

```
ProfilesManager.getInstance().getProfiles()
```

In questo modo viene 'ritornata' un'arraylist che contiene tutti gli oggetti `Profile`.

La lista viene, poi, trasformata in uno stream e ordinata in modo decrescente (utilizzando un comparatore) in base al numero delle vittorie di ciascun profilo.

```
Comparator.comparing(Profile::getNumberOfWins).reversed()
```

Successivamente, lo stream viene limitato ai primi 5 profili attraverso il metodo `limit(5)` che vengono raccolti in una nuova arraylist tramite `collect(Collectors.toList())`. Infine, la lista viene assegnata alla variabile `scores`, che rappresenta i profili aggiornati con i 5 punteggi migliori.

I Utilizzo dei design pattern

I Observer observable pattern

Questo design pattern viene utilizzato per la musica a fine gioco.

La classe `PointManager` nel model estende la classe `Osservable`, mentre `MusicManager` nella view implementa l'interfaccia `Observer`.

Se il giocatore vince (è nella lista dei vincitori), il metodo `didPlayerWin()` 'ritorna' il valore booleano `true` che servirà al metodo `update()` nella classe `MusicManager` e verrà riprodotto il suono della vittoria.

Se il giocatore perde, `didPlayerWin()` 'ritornerà' `false` e verrà riprodotto il suono della sconfitta.

I Singleton pattern

E' stato utilizzato per tutte le classi che devono essere istanziate una sola volta.

Tale approccio è stato scelto per garantire che le classi, utilizzate in diversi punti del programma, siano facilmente accessibili tramite il metodo `getInstance()`.

Un esempio significativo è la classe `CardManager`. Quest'ultima deve essere istanziata una sola volta, in quanto non avrebbe senso avere più di un'istanza di `CardManagerView` all'interno del gioco.

Grazie al singleton, quindi, è possibile garantire un'unica istanza centralizzata, semplificando l'accesso e migliorando la coerenza dei dati.

Inoltre, il pattern contribuisce a mantenere il codice più ordinato ed evita il rischio di duplicazioni o incongruenze.

getInstance()

Questo metodo *statico* fornisce un punto di accesso globale all'unica istanza della classe.

Il metodo verifica se la variabile statica `instance`, che contiene l'istanza della classe, è `null`.

In tal caso, significa che non è ancora stata creata un'istanza di `CardManagerView`.

Se l'istanza non esiste, il metodo la crea invocando il costruttore privato della classe.

L'uso di un costruttore privato (non visibile all'esterno della classe) impedisce che altre parti del codice possano creare nuove istanze della classe.

Il metodo restituisce l'istanza, che viene quindi riutilizzata ogni volta che `getInstance()` viene chiamato.

| Gestione degli utenti, nickname...

Le classi `Profile` e `ProfilesManager` nel model si occupano della gestione dei nicknames, livelli, avatar e conteggio dei risultati delle partite.

ProfilesManager

Provvede all'aggiunta, al salvataggio, alla lettura e alla selezione dei profili.

I metodi `nextProfile()` e `previousProfile()` hanno il compito di cambiare il profilo in fase di selezione, aggiornando l'indice del profilo corrente.

Inoltre, dispone di un metodo (`defaultProfiles()`) che aggiunge dei profili di default nel caso in cui la lista dei profili sia vuota.

Profile

Rappresenta un profilo utente nel gioco, che include il nome, il numero di vittorie e di partite giocate, l'indice dell'avatar e il livello.

Implementa l'interfaccia `Serializable`, utilizzata per abilitare la serializzazione e la deserializzazione degli oggetti, permettendo di trasformare un oggetto in un file su disco (serializzazione) e di ricostruire l'oggetto originale a partire da quel file (deserializzazione). In questo caso, permette di salvare su disco i profili utente, in particolare salva il nickname, il numero di vittorie, il numero di partite giocate e l'indice dell'avatar.

La classe `Profile` si serve dei metodi `read()` e `write()` per leggere e scrivere il profilo corrente.

Infine, c'è un metodo `updateLevel()` che aggiorna di uno il livello del giocatore ogni 5 partite vinte.

Nella view i profili vengono gestiti dalla classe `SelectProfile` che si occupa della loro rappresentazione grafica.

Per la gestione dell'avatar, la classe prende dal model l'indice dell'immagine nella lista degli avatar.

Nel caso in cui si voglia creare un nuovo profilo, è possibile farlo digitando il nickname (gestito dal controller nella classe `SelectProfileController`) e scegliere il proprio avatar iterando sulla lista delle immagini.

Se il nome già esiste, quest'ultimo verrà colorato di rosso e non sarà possibile avviare la partita.

| Gestione di una partita, dei bot e regole del gioco

| Regole del gioco e vincitori

L'obiettivo principale del gioco è ottenere un blackjack (asso e figura).

Se nessun giocatore possiede un blackjack, si controlla chi ha totalizzato 21 con le proprie carte, altrimenti si controlla chi ha il numero più alto che non superi 21.

L'asso ha doppio valore: normalmente vale 11, ma se si supera 21, allora assumerà 1 come valore.

La classe `PointManager` si occupa di stabilire chi sia il vincitore.

Inserisce in una lista il/i vincitore/i che verranno poi (nella view) mostrati a schermo.

Inoltre, ha un metodo `checkIfDealerOver()` che controlla se il mazziere ha superato 21. In tal caso, il banco "sballa" e tutti i giocatori vincono, anche se hanno totalizzato più di 21.

| Gestione turni e classi Entity, Player, Dealer e Bot

La classe astratta `Entity` si occupa della creazione delle varie mani di gioco e dei metodi per l'aggiunta di una carta nella mano di ogni giocatore, metodi che verranno ereditati dalle classi figlie.

Player

Questa classe eredita il metodo `buildHand()` da `Entity` e ha un metodo `hit()` che aggiunge una carta alla mano chiamando il metodo `addCardToHand()` della classe madre. Il giocatore può effettuare il proprio turno utilizzando i bottoni *hit* e *stay*.

Bot e Dealer

Entrambe estendono la classe `Entity` e implementano l'interfaccia `BotAction`, implementando il metodo `turn()`.

Per i bot il turno finisce quando essi ottengono una mano con un valore uguale o superiore a 21. Il mazziere, invece, termina il proprio turno quando ottiene un valore maggiore uguale a 17.

Appena il dealer termina il turno, viene dichiarato il/i vincitore/i e il gioco termina.

I bot sono contenuti in un'arraylist di tipo `Bot`, che viene inizializzata nel menù principale del gioco, quando l'utente decide contro quanti giocatori artificiali vuole giocare (massimo 3).

| MusicManager

Questa classe gestisce la musica e i suoni.

I metodi principali sono `playPlayingSong()`, `playMenuSong()`, `winningSong()` e `losingSong()`, che servono per riprodurre rispettivamente la canzone durante la partita, la canzone del menù, il suono della vittoria e il suono della sconfitta.

I quattro metodi funzionano allo stesso modo, quindi per spiegare come funzionano possiamo prendere come esempio `playPlayingSong()`.

La canzone viene riprodotta in loop finché non viene fermata (per esempio quando il gioco viene messo in pausa).

All'inizio del metodo, viene effettuata una verifica per accertarsi che `playingSong` non sia stato già inizializzato.

Se `playingSong` è `null`, significa che la canzone non è ancora stata caricata e quindi il

programma procederà con il caricamento del file audio.

Durante il caricamento del file audio, il metodo gestisce anche diverse possibili eccezioni (`FileNotFoundException` , `IOException` , `UnsupportedAudioFileException` e `LineUnavailableException`).

Infine, il metodo avvia la riproduzione del brano tramite `playingSong.start()` .